

Deep Sequence Modelling and Transformers

M.I. Capel

ETS Ingenierías Informática y
Telecomunicación
Departamento de Lenguajes y Sistemas Informáticos
Universidad de Granada
Email: manuelcapel@ugr.es
<http://lsi2.ugr.es/mcapel/>

17 de marzo de 2026
Máster Universitario en Desarrollo de Software



- 1 Self-attention fundamentals
- 2 Scaling and stability
- 3 Transformer block
- 4 Training
- 5 Pytorch attention implementation
- 6 Autoregressive generation

Sequence Modelling: What Do We Want?

Given an input sequence of tokens

$$(w_1, w_2, \dots, w_t),$$

we are often interested in one of three fundamental tasks:

- 1 **Sequence labeling**
- 2 **Sequence classification**
- 3 **Autoregressive generation**

These three tasks define different learning objectives over the same basic type of input: an ordered sequence.

Why Sequence Modelling Matters

Many real-world problems are naturally sequential:

- **Language**: words or characters in a sentence
- **Time series**: values observed over time
- **IoT streams**: sensor events arriving sequentially
- **Logs and traces**: ordered system events
- **Biological sequences**: DNA, proteins

Key idea:

A sequence model must exploit the fact that the order of the elements matters.

Sequence Labeling

Definition: assign one output label to each input token.

Example:

(John, lives, in, Granada)

(PER, O, O, LOC)

Typical applications:

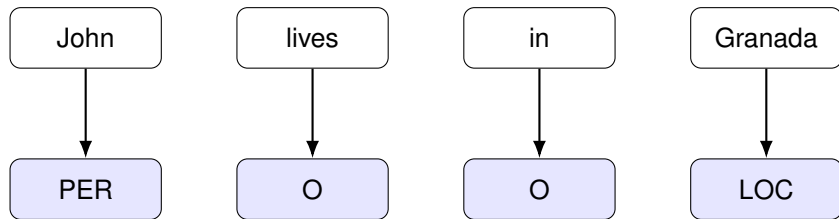
- Named Entity Recognition (NER)
- Part-of-Speech tagging
- Chunking
- Event tagging in logs

Output structure:

$$y_1, y_2, \dots, y_t$$

with one label per position.

Sequence Labeling: Intuition



Each token is processed in context, but the model emits a label for every position.

Sequence Classification

Definition: assign one label to the whole sequence.

Example:

(This course is excellent) $\rightarrow$ Positive

Typical applications:

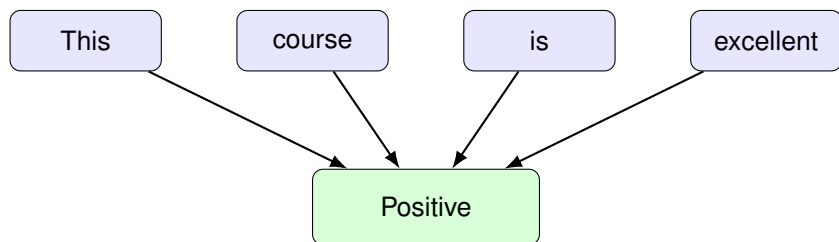
- Sentiment analysis
- Spam detection
- Topic classification
- Intent detection
- Fault classification from temporal traces

Output structure:

y

that summarizes the full sequence.

Sequence Classification: Intuition



The model must aggregate information from the whole sequence into a single decision.

Autoregressive Generation

Definition: predict the next token repeatedly from previous tokens.

Given:

$$(w_1, w_2, \dots, w_t)$$

the model predicts:

$$w_{t+1}$$

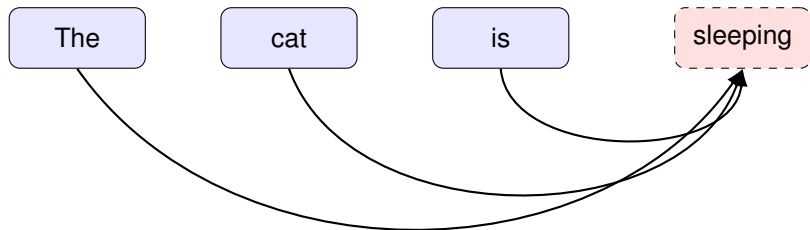
Then the new token is appended and the process is repeated:

$$(w_1, w_2, \dots, w_t, w_{t+1}) \rightarrow w_{t+2}$$

Typical applications:

- Text generation
- Language modelling
- Code completion
- Time-series forecasting

Autoregressive Generation: Intuition



Example:

(The, cat, is) $\rightarrow$ sleeping

Training objective:

predict w_{i+1} from $(w_1, \dots, w_i)$

Comparing the Three Tasks

Task	Input	Output	Example
Sequence labeling	$w_1, \dots, w_t$	$y_1, \dots, y_t$	NER, POS tagging
Sequence classification	$w_1, \dots, w_t$	y	sentiment, topic
Autoregressive generation	$w_1, \dots, w_t$	w_{t+1}	text generation

Same input type, different prediction objective.

A Unifying View

The same sequence

$$(w_1, w_2, \dots, w_t)$$

can support different tasks depending on the output design.

- If we predict one label per token → **sequence labeling**
- If we predict one label for the whole input → **sequence classification**
- If we predict the next token → **autoregressive generation**

Main takeaway

What changes across tasks is not the sequential nature of the input, but the learning objective imposed on the model.

Why This Matters for Transformers

Transformers are especially powerful because they can be adapted to all three tasks:

- **Sequence labeling**: contextual representation at every position
- **Sequence classification**: global sequence representation
- **Autoregressive generation**: next-token prediction with causal masking

This is why transformers have become a general architecture for sequence modelling.

Takeaway

Three core sequence modelling tasks:

- ❶ **Sequence labeling** — one output per token
- ❷ **Sequence classification** — one output for the whole sequence
- ❸ **Autoregressive generation** — predict the next token repeatedly

Fundamental question:

How do transformers represent tokens and model their interactions through self-attention?

From Tokens to Vectors

A transformer does not process words or characters directly.

It receives:

$$w_1, w_2, \dots, w_t$$

where each w_i is a token identifier.

These token IDs are mapped to vectors:

$$w_i \mapsto x_i \in \mathbb{R}^k$$

Key idea

A token is first converted into a dense numerical representation called an **embedding**.

Embedding layer

Map each token id to a vector:

$$w_i \mapsto x_i \in \mathbb{R}^k$$

An embedding layer is a matrix:

$$E \in \mathbb{R}^{|V| \times k}, \quad x_i = E[w_i]$$

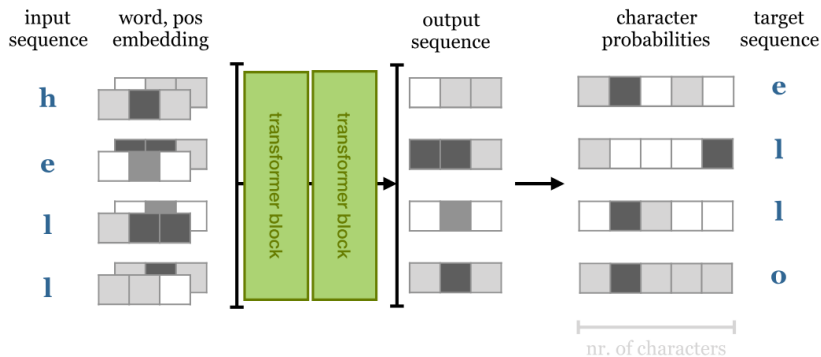


Figure: Character level transformer to predict the next character in a sequence

The Embedding Layer

An embedding layer is a learnable matrix:

$$E \in \mathbb{R}^{|V| \times k}$$

where:

- $|V|$ = vocabulary size
- k = embedding dimension

The embedding of token w_i is the corresponding row of E :

$$x_i = E[w_i]$$

So the sequence

$$(w_1, w_2, \dots, w_t)$$

becomes

$$(x_1, x_2, \dots, x_t), \quad x_i \in \mathbb{R}^k$$

Embedding Layer: Lookup Intuition

Embedding matrix lookup



An embedding layer behaves like a learnable lookup table.

Example: Character-Level Modelling

Suppose the vocabulary is:

$$V = \{a, b, c, \dots, z, \text{space}, \dots\}$$

Then each character is mapped to an integer:

$$a \rightarrow 0, \quad b \rightarrow 1, \quad c \rightarrow 2, \dots$$

and then to a vector:

$$0 \rightarrow x_a, \quad 1 \rightarrow x_b, \quad 2 \rightarrow x_c$$

For a character-level transformer, the input sequence

$$(h, e, l, l, o)$$

is converted into a sequence of vectors:

$$(x_h, x_e, x_l, x_l, x_o)$$

What Does an Embedding Mean?

An embedding vector does *not* usually have an interpretable human meaning.

For example:

$$x_{\text{cat}} = (0.8, 0.3, -0.2, \dots)$$

does not mean:

- feature 1 = animalness
- feature 2 = domesticity

Important idea

Embeddings are learned numerical coordinates that help the model solve the task.

What matters is mainly:

- relative geometry,
- dot products,
- distances and directions,
- infinitely many correct embeddings exist and the model behaves identically as long as dot products are preserved.

Embedding Geometry

If two tokens are useful in similar contexts, training may place them in similar regions of the embedding space.

This means that:

$$x_i^\top x_j$$

may become large when the model learns that two tokens are related for the task.

Typical intuition:

- similar direction $\rightarrow$ large dot product
- orthogonal vectors $\rightarrow$ weak relation
- opposite direction $\rightarrow$ negative relation

This geometric structure is learned automatically by gradient descent.

From a Sequence of Tokens to a Matrix

For a sequence of length t , stack the embeddings row by row:

$$X \in \mathbb{R}^{t \times k}$$

That is,

$$X = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_t \end{bmatrix}$$

This matrix is the numerical input to the transformer.

Transformers operate on a matrix of token representations, not on raw words.

A Fundamental Problem

Suppose we have the two sequences:

`(Mary, greets, Susan)`

and

`(Susan, greets, Mary)`

They contain the same tokens, but the meaning is different.

If a model only looks at token embeddings and ignores position, then order information can be lost.

Problem

Self-attention by itself does not encode token order.

Why do we need positional information?

Self-attention by itself is permutation-equivariant:

$$(x_1, x_2, \dots, x_t) \rightarrow (y_1, y_2, \dots, y_t)$$

If we permute inputs, outputs permute the same way.

Transformers add **positional encodings** (learned or fixed):

$$\tilde{x}_i = x_i + p_i$$

```
1 sequence of tokens: "Mary greets Susan"
```

```
2 -----  
3 Positional encoding for token Susan:  
4 position i
```

```
5 0 ..... (0.00, 0.0)  
6 1 ..... (0.00, 0.2)  
7 2 ..... (0.85, 0.0)
```


Why Positional Information Is Needed

In self-attention, each token can attend to all tokens in the sequence.

But the attention mechanism itself is based on pairwise comparisons between vectors.

If we permute the input sequence, the outputs are permuted accordingly.

So, by itself, self-attention is **permutation-equivariant**.

This means:

- it sees *which* tokens are present,
- but not inherently *where* they are.

Positional Encoding: Main Idea

To inject order information, transformers add a positional vector p_i to each token embedding x_i :

$$\tilde{x}_i = x_i + p_i$$

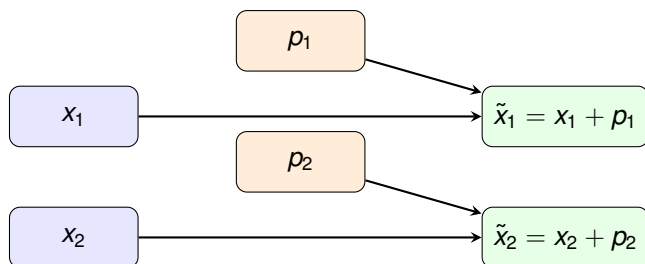
Now each input representation contains:

- token identity information
- position information

So instead of feeding x_i into the model, we feed:

$$\tilde{x}_i$$

Positional Encoding: Visual Intuition



Each token embedding is enriched with a vector that represents its position in the sequence.

Example: Same Word, Different Position

The same token can appear at different positions:

the

Its embedding may be the same base vector:

x_{the}

But position changes the final input to attention:

$$\tilde{x}_i = x_{\text{the}} + p_i \quad \tilde{x}_j = x_{\text{the}} + p_j$$

Therefore, the model can distinguish:

- the at the beginning,
- the in the middle,
- the near the end.

Two Main Approaches to Positional Encoding

There are two common ways to define p_i :

- 1 **Fixed positional encodings** predefined mathematical vectors
- 2 **Learned positional encodings** trainable vectors, like embeddings

Both aim to tell the model where each token occurs in the sequence.

Learned Positional Embeddings

In this approach, each position has a trainable vector:

$$p_0, p_1, p_2, \dots$$

These vectors are learned jointly with the rest of the model.

Advantages:

- simple to implement
- flexible
- often effective in practice

Limitation:

- tied to the maximum sequence length seen during training

Fixed Positional Encodings

Instead of learning p_i , we can define it analytically.

The classical transformer uses sinusoidal encodings:

$$PE(i, 2m) = \sin\left(\frac{i}{10000^{2m/k}}\right)$$

$$PE(i, 2m + 1) = \cos\left(\frac{i}{10000^{2m/k}}\right)$$

where:

- i is the position
- m is the coordinate index
- k is the embedding dimension

These functions generate a different positional pattern for each token position.

Visual Intuition of Sinusoidal Positional Encodings

For each position i , the Transformer builds a vector

$$PE(i) = (PE(i, 0), PE(i, 1), \dots, PE(i, k-1))$$

using alternating sine and cosine functions.

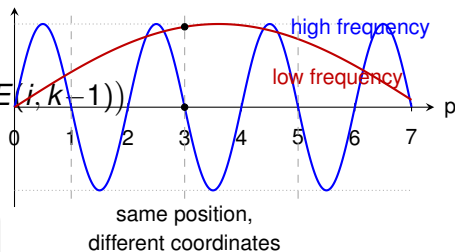
Key intuition

Different dimensions oscillate at different frequencies.

- low-index dimensions vary rapidly
- higher-index dimensions vary more slowly
- each position gets a unique multi-scale pattern

This helps the model encode both:

- absolute position
- relative distance



Each dimension uses a sinusoid with a different wavelength.

Why Sinusoidal Encodings Are Interesting

Sinusoidal encodings provide:

- a smooth representation of position,
- multiple frequencies,
- a way to represent relative shifts geometrically.

Intuition:

- low-frequency components capture coarse position
- high-frequency components capture finer distinctions

They also allow extrapolation to longer sequences more naturally than purely learned position tables.

Mini Example

Consider the sequence:

(Mary, greets, Susan)

Suppose:

x_{Mary} , x_{greets} , x_{Susan}

are token embeddings, and

p_1 , p_2 , p_3

are positional vectors.

Then the actual transformer inputs are:

$$\tilde{x}_1 = x_{\text{Mary}} + p_1$$

$$\tilde{x}_2 = x_{\text{greets}} + p_2$$

$$\tilde{x}_3 = x_{\text{Susan}} + p_3$$

Now both token identity and token order are represented.

Main Takeaways

- 1 Tokens are mapped to dense vectors called **embeddings**.
- 2 The embedding layer is a learnable lookup matrix:

$$E \in \mathbb{R}^{|V| \times k}$$

- 3 A transformer works on sequences of vectors, not raw words.
- 4 Self-attention alone does not encode order.
- 5 Positional encodings solve this by adding a position vector:

$$\tilde{x}_i = x_i + p_i$$

Transition to the Next Topic

We now have a sequence of enriched vectors:

$$\tilde{x}_1, \tilde{x}_2, \dots, \tilde{x}_t$$

The next question is:

Fundamental

How does each token decide which other tokens matter?

This leads directly to the **self-attention** mechanism.

Why Context Matters

Consider the token:

bank

Its meaning changes depending on context:

The bank approved the loan

They sat by the bank of the river

A fixed representation for `bank` is not enough.

Problem

A token should not be represented in isolation only. Its representation must depend on the other tokens in the sequence.

From Static to Contextual Representations

Suppose the sequence is:

$$(w_1, w_2, \dots, w_t)$$

After the embedding layer, each token has a vector:

$$x_1, x_2, \dots, x_t$$

These are **input representations**. But what we want is:

$$y_1, y_2, \dots, y_t$$

where each y_i is a **contextual representation**.

That means:

$$y_i \text{ should depend on } x_i \text{ and also on } x_1, \dots, x_t$$

Core Idea of Self-Attention

For each token position i , we build a new vector y_i as a weighted combination of all input vectors:

$$y_i = \sum_{j=1}^t w_{ij} x_j$$

where:

- x_j are the input token vectors,
- w_{ij} indicates how much token i attends to token j ,
- the weights satisfy:

$$w_{ij} \geq 0, \quad \sum_{j=1}^t w_{ij} = 1$$

So each output vector is a weighted average of the input vectors.

Interpretation of the Formula

$$y_i = \sum_{j=1}^t w_{ij} x_j$$

Interpretation:

- token i looks at all tokens in the sequence,
- it decides which ones are more relevant,
- it builds its new representation by aggregating those tokens.

Key intuition

Self-attention lets each token gather information from the whole sequence.

How Are the Weights Computed?

We need a mechanism to determine how relevant token j is for token i .

Basic self-attention uses a similarity score:

$$s_{ij} = x_i^\top x_j$$

If x_i and x_j are well aligned, s_{ij} is large.

Then these scores are normalized:

$$w_{ij} = \text{softmax}_j(s_{ij})$$

So the final output is:

$$y_i = \sum_{j=1}^t w_{ij} x_j$$

Self-attention operation

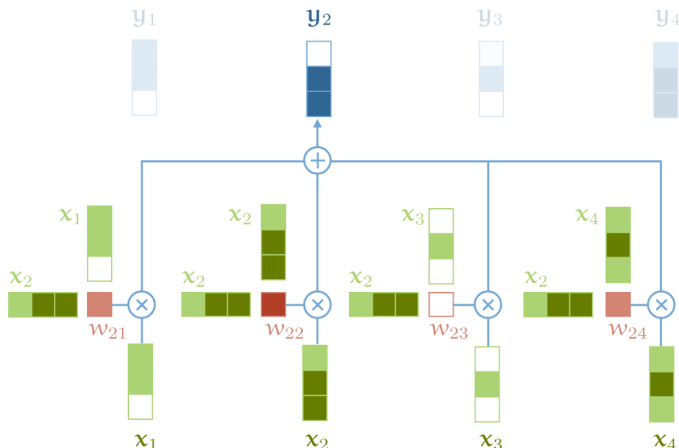


Figure: Representation of the fundamental operation of any transformer architecture

Key question of a self-attention architecture:

How does a token decide *which* other tokens matter?

Basic Self-Attention Equations

The output vector

$$y_i = \sum_j w_{ij} x_j$$

is a convex combination of the input vectors, because:

$$w_{ij} \geq 0, \quad \sum_j w_{ij} = 1$$

This means:

- self-attention redistributes and combines existing information,
- it does not invent arbitrary new directions at this step,
- it reweights the sequence according to contextual relevance.

A Full Sequence View

For each position i , we compute a different set of weights:

$$w_{i1}, w_{i2}, \dots, w_{it}$$

Therefore, each output vector y_i is different:

$$y_1, y_2, \dots, y_t$$

Even though all positions use the same mechanism, each token attends differently because its input vector differs.

The Role of Softmax

Given scores:

$$s_{i1}, s_{i2}, \dots, s_{it}$$

the softmax transforms them into normalized weights:

$$w_{ij} = \frac{e^{s_{ij}}}{\sum_{u=1}^t e^{s_{iu}}}$$

Properties:

- $w_{ij} \geq 0$
- $\sum_j w_{ij} = 1$
- larger scores receive larger weights

So softmax turns arbitrary real-valued scores into a probability-like attention distribution.

Softmax: Why Not Use Raw Scores?

Raw scores s_{ij} can be:

- negative,
- positive,
- unbounded,
- not normalized.

This makes them difficult to interpret directly as attention strengths.

Softmax solves this because it produces:

- nonnegative values,
- normalized weights,
- smooth competition among tokens.

Softmax: Main Property

A useful property is:

$$\text{softmax}(\mathbf{s}) = \text{softmax}(\mathbf{s} + \mathbf{c})$$

for any constant c added to all components.

This means:

- only relative differences matter,
- attention depends on score comparisons, not absolute magnitude shifts.

So the model cares about which tokens are more aligned, not about an arbitrary common offset.

Dot Products as Similarity

The basic score is:

$$s_{ij} = x_i^\top x_j$$

Recall:

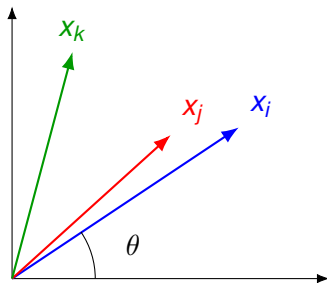
$$x_i^\top x_j = \|x_i\| \|x_j\| \cos(\theta)$$

where θ is the angle between the vectors.

So:

- similar directions $\Rightarrow$ large positive score,
- orthogonal vectors $\Rightarrow$ score near zero,
- opposite directions $\Rightarrow$ negative score.

Geometric Interpretation



If vectors are aligned, their dot product is large, so the corresponding attention weight tends to be larger.

Why This Is Powerful

Self-attention allows the representation at one position to incorporate information from distant positions.

That means:

- long-range dependencies can be modeled directly,
- each token can access the whole sequence in one layer,
- interactions are highly parallelizable.

This is one of the main reasons transformers became so successful.

Toy example

"The cat walks on street":

- Let's take embeddings as 2D vectors

	words in sequence	vector
1		
2	the	(0.1, 0.0)
3	cat	(1.0, 0.9)
4	walks	(0.9, 1.0)
5	on	(0.0, 0.1)
6	street	(0.8, 0.7)

- Dot products with token 'cat':

$$s_{10} = v_{cat} \cdot v_{the} = (1.0, 0.9) \cdot (0.1, 0.0)^T = 0.1$$

$$s_{11} = v_{cat} \cdot v_{cat} = (1.0, 0.9) \cdot (1.0, 0.9)^T = 1.81$$

$$s_{12} = v_{cat} \cdot v_{walks} = (1.0, 0.9) \cdot (0.9, 1.0)^T = 1.80$$

$$s_{13} = v_{cat} \cdot v_{on} = (1.0, 0.9) \cdot (0.0, 0.1)^T = 0.09$$

$$s_{14} = v_{cat} \cdot v_{street} = (1.0, 0.9) \cdot (0.8, 0.7)^T = 1.43$$

Toy example-ii

$$\text{softmax}_u(s_{1,u}) = \text{softmax}_u(s_{cat,u}) = \left(\frac{e^{s_{cat,u}}}{\sum_j e^{s_{cat,j}}} \right)$$

$e^{\frac{s_{10}}{\sqrt{2}}}$	1.1073	$\text{softmax}_0(\frac{s_{10}}{\sqrt{(2)}})$	0.0916
$e^{\frac{s_{11}}{\sqrt{2}}}$	3.5962	$\text{softmax}_1(\frac{s_{11}}{\sqrt{(2)}})$	0.2975
$e^{\frac{s_{12}}{\sqrt{2}}}$	3.5709	$\text{softmax}_2(\frac{s_{12}}{\sqrt{(2)}})$	0.2954
$e^{\frac{s_{13}}{\sqrt{2}}}$	1.0657	$\text{softmax}_3(\frac{s_{13}}{\sqrt{(2)}})$	0.0882
$e^{\frac{s_{14}}{\sqrt{2}}}$	2.7488	$\text{softmax}_4(\frac{s_{14}}{\sqrt{(2)}})$	0.2273
Sum	12.0889	Sum	1.0000

$$w_{ij} = \text{softmax}_j \left(\frac{s_{ij}}{\sqrt{d}} \right)$$

Matrix Form of Basic Self-Attention

Stack the input vectors into:

$$X \in \mathbb{R}^{t \times k}$$

Then the score matrix is:

$$S = XX^{\top}$$

Each entry is:

$$S_{ij} = x_i^{\top} x_j$$

Apply softmax row-wise:

$$W = \text{softmax}(S)$$

Then the output matrix is:

$$Y = WX$$

This is the matrix form of basic self-attention.

What Does the Score Matrix Mean?

$$S = XX^T$$

Interpretation:

- row i tells us how token i compares to all tokens,
- column j tells us how all tokens compare to token j ,
- diagonal entries S_{ii} are self-similarities.

After softmax, each row becomes an attention distribution over the sequence.

Visualizing the Attention Matrix

$$W = \begin{bmatrix} w_{11} & w_{12} & w_{13} & w_{14} \\ w_{21} & w_{22} & w_{23} & w_{24} \\ w_{31} & w_{32} & w_{33} & w_{34} \\ w_{41} & w_{42} & w_{43} & w_{44} \end{bmatrix}$$

Each row sums to 1:

$$\sum_j w_{ij} = 1$$

Row i tells us:

How much token i attends to every token j

Self-Attention vs Recurrence

In recurrent models:

- information is propagated step by step,
- long-range interactions are indirect,
- computation is sequential.

In self-attention:

- each token can interact directly with every token,
- long-range interactions are explicit,
- computation is parallel across positions.

RNN intuition vs attention intuition

- RNN: build state left-to-right; hard to parallelize.

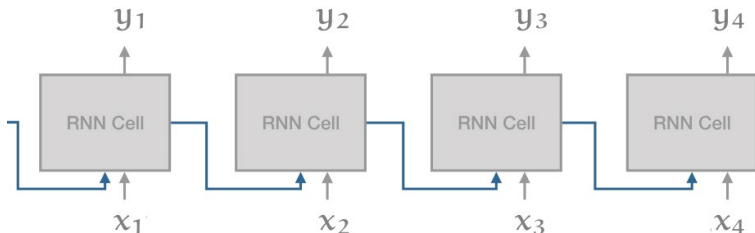


Figure: Unrolled RNN (usually an LSTM or a GRU), previous state-of-the-art architecture

- **Self-attention:** let each position look at all positions; highly parallel.

Why Do We Need Embeddings?

Token IDs are just symbols:

cat $\rightarrow$ 17, dog $\rightarrow$ 42

These integers do *not* contain useful geometric information.

A neural model needs vector representations so that it can:

- compare tokens numerically,
- compute dot products,
- learn similarities,
- transform representations through linear maps.

Therefore, token IDs are mapped to dense vectors in $\mathbb{R}^k$.

Why It Helps with Long Dependencies

Suppose token x_1 influences token x_t .

In an RNN, the signal must pass through many recurrent steps.

In self-attention, x_t can directly assign a large weight to x_1 :

$$y_t = \sum_{j=1}^t w_{tj} x_j$$

with w_{t1} potentially large.

So long-distance interactions are modeled more directly.

Important reflections on the **attention** mechanism

Embeddings and geometric equivalence

- Embeddings are typically initialized at random.
- Training updates them through gradient descent.
- The loss function encourages dot products to become useful for the task.
- The coordinates settle into whatever values minimize the loss.
- Only geometric relationships (e.g., distances, angles, dot products) matter.
- Embedding coordinates are learned parameters, just like the rest of the model.

Important reflections on the **attention** mechanism–ii

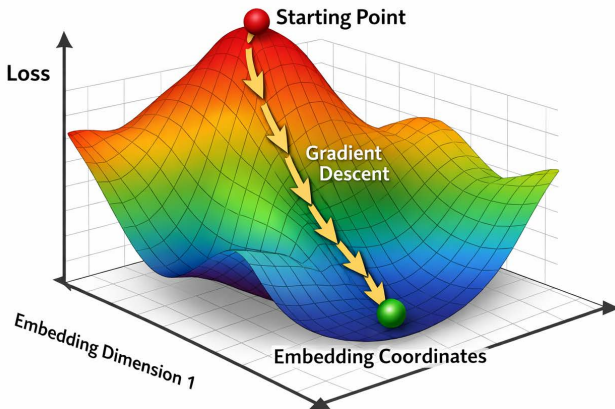


Figure: Vector optimization with gradient descent

But There Is a Limitation

In basic self-attention, the same vector x_i plays three roles simultaneously:

- it decides whom to attend to,
- it is compared as a candidate token,
- it provides the content being copied.

This is conceptually restrictive.

Limitation

The same vector is used for querying, matching, and content delivery.

Three Roles Hidden in One Vector

Ideally, a token should have distinct representations for:

- **Query**: what am I looking for?
- **Key**: what do I contain?
- **Value**: what information do I provide?

Basic attention does not separate these roles.

Transformers solve this using learned linear projections.

Motivation for Queries, Keys and Values

Instead of using x_i directly for everything, transformers learn:

$$q_i = W_Q x_i, \quad k_i = W_K x_i, \quad v_i = W_V x_i$$

Then scores are computed with:

$$q_i^\top k_j$$

and outputs aggregate:

$$v_j$$

This is the next major conceptual step after basic self-attention.

Bridge to the Full Transformer Formula

The full transformer attention mechanism becomes:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right) V$$

Today we have focused on the simpler precursor:

$$y_i = \sum_j w_{ij} x_j, \quad w_{ij} = \text{softmax}_j(x_i^\top x_j)$$

The basic idea is already present:

- compare tokens,
- normalize scores,
- compute weighted combinations.

Concept Check

- 1 Why is a fixed embedding not enough for ambiguous words?
- 2 Why do we normalize scores with softmax?
- 3 What does a large dot product mean geometrically?
- 4 Why is y_i a contextual representation?
- 5 What is the main limitation of using x_i directly for everything?

Suggested Exercise

Given:

$$x_1 = (1, 0), \quad x_2 = (1, 1), \quad x_3 = (0, 1)$$

Let's compute:

- 1 the score vector for token 2,
- 2 the corresponding softmax weights,
- 3 the output vector y_2 .

This exercise reinforces:

- dot products,
- softmax normalization,
- weighted sums.

Takeaways

- Self-attention is the mechanism that lets tokens interact.
- Each token builds its representation from the whole sequence.
- Similarity scores determine relevance.
- Softmax turns scores into normalized attention weights.
- The result is a contextual vector at each position.
- Basic self-attention motivates the introduction of Q/K/V.

Transition to the Next Block

Next question:

How do transformers separate the roles of querying, matching and value aggregation?

Next topic:

Queries, Keys, Values, and scaled dot-product attention

Learning Goals

- grasp the limitation of basic self-attention,
- define the roles of Queries, Keys and Values,
- write the projected attention equations,
- derive the matrix form of attention,
- why scaling by $\sqrt{d_k}$ is needed?,
- interpret scaled dot-product attention geometrically,
- understand how Q/K/V improve expressiveness over raw embeddings.

Limitation of Basic Self-Attention

In the basic formulation, the same vector x_i must do three jobs at once:

- decide what it is looking for,
- act as something that can be matched by others,
- carry the content that will be aggregated.

Problem

A single vector is being used simultaneously as: query, matching key, and content value.

This is restrictive, because these roles are conceptually different.

Three Conceptual Roles

For each token, it is useful to distinguish:

- **Query:** What am I looking for?
- **Key:** What do I contain that others may want?
- **Value:** If someone attends to me, what information do I provide?

These three roles motivate the introduction of three learned projections.

Queries, Keys and Values

Given an input token representation $x_i \in \mathbb{R}^k$, transformers define:

$$q_i = W_Q x_i$$

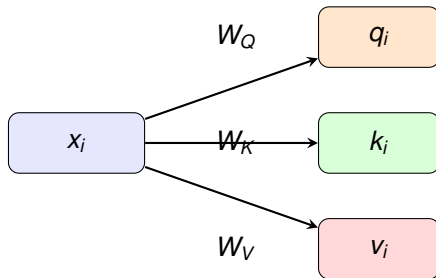
$$k_i = W_K x_i$$

$$v_i = W_V x_i$$

where:

- W_Q projects into query space,
- W_K projects into key space,
- W_V projects into value space.

Visual Intuition



A single token representation is projected into three different spaces.

Why Learned Projections Help

With learned projections:

- the model can learn what kinds of patterns should be sought,
- it can learn a different representation for matching,
- it can learn a different representation for the content that is transferred.

So instead of comparing raw embeddings directly, the model compares transformed representations that are specialized for the task.

Projected Attention Scores

Once Queries and Keys are defined, the score between token i and token j becomes:

$$s_{ij} = q_i^\top k_j$$

Then attention weights are computed with softmax:

$$w_{ij} = \text{softmax}_j(s_{ij})$$

Finally, the output is a weighted sum of the Value vectors:

$$y_i = \sum_{j=1}^t w_{ij} v_j$$

This is the central upgrade from basic self-attention.

Basic vs Projected Attention

Basic attention	Projected attention	Role separation
$x_i^\top x_j$	$q_i^\top k_j$	yes
aggregate x_j	aggregate v_j	yes
same vector for all roles	different spaces	more expressive

Projected attention is more flexible because matching and content aggregation are no longer forced to use the same representation.

Interpretation of the Three Spaces

- **Query space**: describes the kind of information a token is looking for.
- **Key space**: describes the features a token offers for matching.
- **Value space**: describes the information that will actually be passed to other tokens.

Key idea

Matching happens in Query/Key space, but aggregation happens in Value space.

The Output Formula with Q/K/V

For each position i :

$$s_{ij} = q_i^\top k_j$$

$$w_{ij} = \text{softmax}_j(s_{ij})$$

$$y_i = \sum_{j=1}^t w_{ij} v_j$$

Notice the distinction:

- q_i decides whom to attend to,
- k_j determines whether token j is relevant,
- v_j provides the content that is copied into the output.

Toy Example: Role Interpretation

Suppose the sequence is:

(The, cat, walks, home)

When processing `cat`, the model may:

- use its Query to look for syntactically or semantically related tokens,
- compare with the Keys of other words,
- aggregate the Values of the most relevant tokens.

The point is not that the model knows grammar explicitly, but that it can learn useful matching patterns through the projections.

From Vectors to Matrices

For a sequence of length t , stack token representations row-wise:

$$X \in \mathbb{R}^{t \times k}$$

Then define:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

If:

$$W_Q, W_K \in \mathbb{R}^{k \times d_k}, \quad W_V \in \mathbb{R}^{k \times d_v}$$

then:

$$Q, K \in \mathbb{R}^{t \times d_k}, \quad V \in \mathbb{R}^{t \times d_v}$$

Matrix Form of Attention

Once we have Q , K and V :

$$QK^{\top} \in \mathbb{R}^{t \times t}$$

This gives all pairwise Query–Key similarities.

Apply softmax row-wise:

$$W = \text{softmax}(QK^{\top})$$

Then compute:

$$Y = WV$$

So the matrix form is:

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^{\top})V$$

Shape Check

It is useful to track dimensions carefully.

$$Q \in \mathbb{R}^{t \times d_k}, \quad K^\top \in \mathbb{R}^{d_k \times t}$$

Therefore:

$$QK^\top \in \mathbb{R}^{t \times t}$$

This is the attention score matrix.

Then:

$$W \in \mathbb{R}^{t \times t}, \quad V \in \mathbb{R}^{t \times d_v}$$

Hence:

$$WV \in \mathbb{R}^{t \times d_v}$$

Meaning of the Score Matrix

$$S = QK^{\top}$$

Each entry:

$$S_{ij} = q_i^{\top} k_j$$

Interpretation:

- row i : what token i is looking for across the sequence,
- column j : how strongly token j matches different queries.

After row-wise softmax, each row becomes an attention distribution.

Why Scaling Is Needed

The standard transformer does not use:

$$\text{softmax}(QK^{\top})V$$

Instead, it uses:

$$\text{softmax}\left(\frac{QK^{\top}}{\sqrt{d_k}}\right)V$$

Question:

Why divide by $\sqrt{d_k}$?

The Problem Without Scaling

Suppose query and key vectors have dimension d_k .

If their components have typical size of order 1, then the dot product:

$$q_i^\top k_j = \sum_{\ell=1}^{d_k} q_{i\ell} k_{j\ell}$$

tends to grow with d_k .

That means:

- higher dimensions can produce large raw scores,
- large scores make softmax very peaked,
- very peaked softmax leads to tiny gradients.

Softmax Saturation

If one score is much larger than the others, softmax becomes almost one-hot.

Example:

$$\text{softmax}(1, 2, 1) \approx (0.21, 0.58, 0.21)$$

but:

$$\text{softmax}(10, 20, 10) \approx (0, 1, 0)$$

When softmax becomes too sharp:

- learning becomes less stable,
- gradients can become very small,
- optimization gets harder.

Scaling by $\sqrt{d_k}$

To keep scores in a healthier range, transformers scale them:

$$s_{ij} = \frac{q_i^\top k_j}{\sqrt{d_k}}$$

Then:

$$w_{ij} = \text{softmax}_j \left(\frac{q_i^\top k_j}{\sqrt{d_k}} \right)$$

and:

$$y_i = \sum_j w_{ij} v_j$$

This leads to the standard attention formula:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^\top}{\sqrt{d_k}} \right) V$$

Intuition Behind the Scaling

Scaling does not change the qualitative mechanism.

It simply keeps the magnitudes of the scores under control.

Main intuition

The factor $\sqrt{d_k}$ prevents dot products from becoming too large when the dimensionality increases.

So the attention scores remain informative without pushing softmax into saturation too early.

Why Attention Scores Grow with Dimension

In self-attention we compute similarity scores using a dot product:

$$s_{ij} = q_i^\top k_j$$

where $q_i, k_j \in \mathbb{R}^{d_k}$

The dot product is:

$$q_i^\top k_j = \sum_{\ell=1}^{d_k} q_{i\ell} k_{j\ell}$$

Assume the components of the vectors are approximately:

$$q_{i\ell}, k_{j\ell} \sim \mathcal{N}(0, 1)$$

Then: $\text{Var}(q_i^\top k_j) = d_k$

Consequence

The magnitude of the dot product typically grows like:

$$|q_i^\top k_j| \sim O(\sqrt{d_k})$$

So larger vector dimensions produce larger attention scores.

Why We Divide by $\sqrt{d_k}$

Attention uses softmax to normalize scores:

$$w_{ij} = \text{softmax}_j(s_{ij})$$

If the scores become large, softmax becomes extremely sharp.

Example:

but $\text{softmax}(1, 2, 1) \approx (0.21, 0.58, 0.21)$

$$\text{softmax}(10, 20, 10) \approx (0, 1, 0)$$

This causes:

- almost one-hot attention
- very small gradients
- unstable training

Why We Divide by $\sqrt{d_k}$ -ii

To stabilize the scores, Transformers scale them:

$$s_{ij} = \frac{q_i^\top k_j}{\sqrt{d_k}}$$

Result

The variance of the scores remains approximately constant as the dimensionality increases.

This yields the final attention formula:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

Geometric Interpretation of Attention Scores

The dot product between two vectors can be written as:

$$q_i^\top k_j = \|q_i\| \|k_j\| \cos(\theta_{ij})$$

where

- $\|q_i\|$ and $\|k_j\|$ are vector magnitudes
- θ_{ij} is the angle between the vectors

Geometric meaning

Attention measures how aligned two vectors are.

$$\cos(\theta_{ij}) \approx 1 \quad \Rightarrow \quad \text{strong attention}$$

$$\cos(\theta_{ij}) \approx 0 \quad \Rightarrow \quad \text{weak attention}$$

Geometric Interpretation of Attention Scores—ii

However, the dot product also depends on the vector magnitudes:

$$q_i^\top k_j = \|q_i\| \|k_j\| \cos(\theta_{ij})$$

As dimensionality grows, vector magnitudes tend to increase.

Why scaling helps

Dividing by $\sqrt{d_k}$ keeps the magnitude of scores in a stable range so that attention depends mainly on **alignment (angle)** rather than vector size.

Geometric Perspective on Scaling

Without scaling:

- large dimensions amplify raw alignment scores,
- tiny geometric differences may become numerically exaggerated.

With scaling:

- comparisons remain better conditioned,
- softmax receives scores in a more stable numerical range.

So scaling improves the training dynamics, not the conceptual meaning of attention.

The Full Scaled Dot-Product Attention Formula

The standard transformer attention mechanism is:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^{\top}}{\sqrt{d_k}}\right) V$$

This formula contains four steps:

- 1 project token representations into $Q/K/V$,
- 2 compute pairwise scores with $QK^{\top}$,
- 3 normalize them with softmax,
- 4 aggregate the value vectors with the resulting weights.

Step-by-Step Computational Pipeline

$$X \xrightarrow{W_Q, W_K, W_V} Q, K, V$$

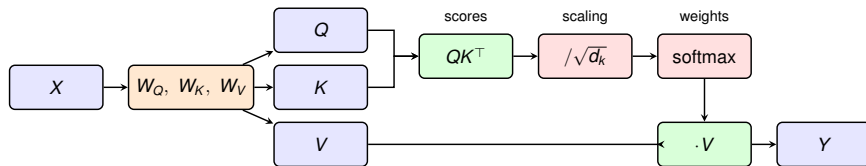
$$QK^{\top} \rightarrow \text{score matrix}$$

$$\frac{QK^{\top}}{\sqrt{d_k}} \rightarrow \text{scaled scores}$$

$$\text{softmax}\left(\frac{QK^{\top}}{\sqrt{d_k}}\right) \rightarrow \text{attention weights}$$

$$\text{softmax}\left(\frac{QK^{\top}}{\sqrt{d_k}}\right) V \rightarrow \text{contextual outputs}$$

Scaled Dot-Product Attention Pipeline



$$Y = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

Worked Symbolic Example

For a token at position i :

$$q_i = W_Q x_i$$

$$k_j = W_K x_j$$

$$v_j = W_V x_j$$

Then compute:

$$s_{ij} = \frac{q_i^\top k_j}{\sqrt{d_k}}$$

Normalize across j :

$$w_{ij} = \text{softmax}_j(s_{ij})$$

Finally:

$$y_i = \sum_{j=1}^t w_{ij} v_j$$

What Has Changed from Basic Attention?

Compared with basic self-attention:

- tokens are no longer compared in the raw embedding space,
- similarity is computed in learned Query/Key space,
- transferred content comes from learned Value space,
- scaling improves numerical stability.

Result

Attention becomes both more expressive and easier to train.

Conceptual Summary

Component	Question it answers	Role
Query q_i	What am I looking for?	attention request
Key k_j	What do I offer?	matchability
Value v_j	What do I provide?	transmitted content

Mini Example for Discussion

If two tokens have similar Keys but very different values:

- will they attract similar attention?
- will they contribute the same information?
- are Queries, Keys and Values different tokens?

Bridge to Implementation

In code, the projections are usually implemented as matrix multiplications:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

Then:

$$\text{scores} = \frac{QK^\top}{\sqrt{d_k}}$$

$$\text{weights} = \text{softmax}(\text{scores})$$

$$Y = \text{weights} \cdot V$$

This is exactly the structure we will later see in PyTorch.

The Embedding Layer

An embedding layer is a learnable matrix

$$E \in \mathbb{R}^{|V| \times K},$$

where $|V|$ is the vocabulary size and K is the embedding dimension. The embedding of token t is the t -th row: $v_t = E_t \in \mathbb{R}^K$. E is initialized randomly and learned during training.

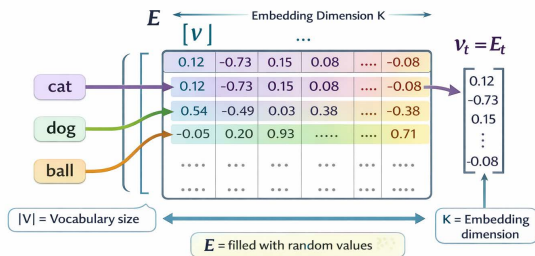


Figure: Embedding matrix as a learnable lookup table

Summary of the Lesson

Core equations:

$$q_i = W_Q x_i, \quad k_i = W_K x_i, \quad v_i = W_V x_i$$

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

Core ideas:

- separate matching from content,
- learn specialized spaces,
- scale scores for stable optimization.

Concept Check

- 1 What limitation of basic self-attention motivates Q/K/V?
- 2 What is the difference between a Key and a Value?
- 3 Why do we compute scores using $q_i^\top k_j$ and not $v_i^\top v_j$?
- 4 Why is the factor $\sqrt{d_k}$ needed?
- 5 What are the dimensions of $QK^\top$ and of the final output Y ?

Takeaways

- Queries, Keys and Values separate three different roles.
- Attention matching happens in Query/Key space.
- Aggregation happens in Value space.
- The matrix formula is:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

- Scaling by $\sqrt{d_k}$ keeps scores in a numerically healthy range.

Takeaways-ii

Properties of self-attention

Dot products increase or decrease because gradient descent mechanically adjust them in whatever direction to lower the loss.

Properties

- The attention weights are computed dynamically from the input.
- The dot-product + softmax operation has no parameters.
- Without positional encoding, self-attention is permutation invariant.

Transition to the Next Block

Next questions:

How does this fit into a complete Transformer block? What comes after attention?

Next topic:

Transformer block structure and training pipeline

Main question

How is scaled dot-product attention embedded inside a full trainable Transformer layer?

Basic views of an embedding in self-attention

Fundamental idea

A token simultaneously asks a question (query), advertises what it contains (key), and carries content (value).

Three roles per token in a sequence

- Query q_i : What am I looking for in other tokens?
- Key k_j : What do I offer that other tokens might look for?
- Value v_j : If someone attends to me, what information do I provide?

Self-attention mechanism in transformers

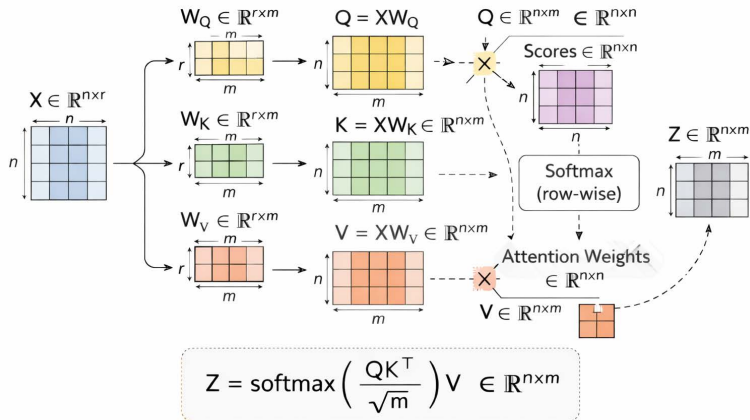


Figure: Self-attention mechanism in Transformers

Basic views of an embedding in self-attention-II

Learned projections and attention weights

Embeddings are projected using learned matrices:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

Attention weights are computed as:

$$w_{ij} = \text{softmax}_j \left(\frac{q_i^\top k_j}{\sqrt{d_k}} \right)$$

The output representation becomes:

$$y_i = \sum_j w_{ij} v_j$$

From Attention to Transformer Block

Scaled dot-product attention gives us:

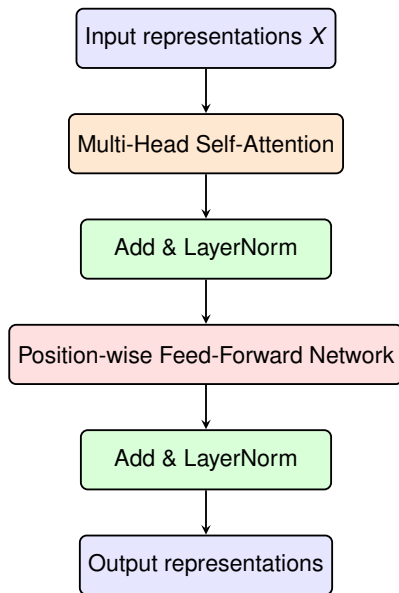
$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

But a Transformer layer contains more than attention.

A standard Transformer block includes:

- 1 multi-head self-attention,
- 2 residual connection,
- 3 layer normalization,
- 4 position-wise feed-forward network,
- 5 another residual connection,
- 6 another layer normalization.

High-Level Transformer Block



Why Multi-Head Attention?

Instead of using a single attention mechanism, Transformers usually use several heads in parallel.

Idea:

- each head learns its own projections,
- different heads can focus on different relations,
- their outputs are combined.

So multi-head attention increases expressive power.

Multi-Head Attention: Concept

For head h :

$$Q^{(h)} = XW_Q^{(h)}, \quad K^{(h)} = XW_K^{(h)}, \quad V^{(h)} = XW_V^{(h)}$$

$$\text{head}^{(h)} = \text{softmax}\left(\frac{Q^{(h)}K^{(h)\top}}{\sqrt{d_k}}\right) V^{(h)}$$

Then concatenate the heads and project again:

$$\text{MHA}(X) = \text{Concat}(\text{head}^{(1)}, \dots, \text{head}^{(H)})W_O$$

This is the standard multi-head attention mechanism.

Why Several Heads Help

Different heads can learn different patterns, for example:

- local syntactic dependencies,
- long-range semantic relations,
- positional interactions,
- task-specific alignments.

Key idea

A single attention head gives one attention pattern. Multi-head attention gives several complementary attention patterns.

Residual Connections

After multi-head attention, the Transformer adds the original input back:

$$X + \text{MHA}(X)$$

This is a residual connection.

Why is it useful?

- improves gradient flow,
- stabilizes deep training,
- helps the model preserve useful information,
- makes optimization easier.

Layer Normalization

After the residual addition, the Transformer applies layer normalization:

$$Z_1 = \text{LayerNorm}(X + \text{MHA}(X))$$

Layer normalization helps by:

- stabilizing activation statistics,
- making optimization more robust,
- reducing internal covariate shift across features.

Residual connections and normalization are crucial for stable deep learning.

The Feed-Forward Network

After attention and normalization, each position is processed by the same small neural network:

$$\text{FFN}(x) = W_2 \sigma(W_1 x + b_1) + b_2$$

where σ is usually ReLU or GELU.

Important:

- the FFN is applied independently at each position,
- the same weights are reused for all positions,
- this adds nonlinear transformation capacity.

Why the Feed-Forward Layer Is Needed

Attention mixes information across positions.

The feed-forward network then:

- transforms each contextual vector nonlinearly,
- increases representational power,
- lets the model reshape features after attention.

So the block alternates:

interaction across positions $\rightarrow$ nonlinear feature transformation

Second Residual + Normalization

After the FFN, the Transformer again adds a residual connection and normalizes:

$$Z_2 = \text{LayerNorm}(Z_1 + \text{FFN}(Z_1))$$

So the full block becomes:

$$Z_1 = \text{LayerNorm}(X + \text{MHA}(X))$$

$$Z_2 = \text{LayerNorm}(Z_1 + \text{FFN}(Z_1))$$

Here, Z_2 is the block output.

Full Transformer Block Equations

A compact summary is:

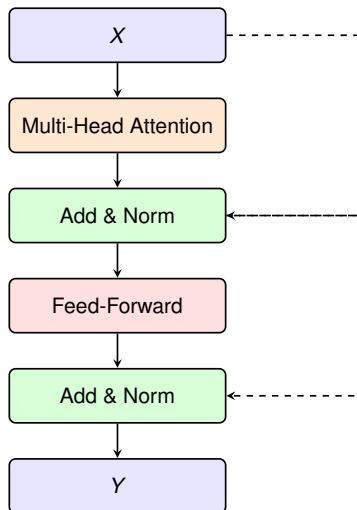
$$Z_1 = \text{LayerNorm}(X + \text{MHA}(X))$$

$$Y = \text{LayerNorm}(Z_1 + \text{FFN}(Z_1))$$

where:

- X is the input sequence representation,
- $\text{MHA}(X)$ captures contextual interactions,
- FFN transforms each position independently,
- Y is the output of the block.

Transformer Block: Visual Pipeline



Stacking Several Blocks

A Transformer encoder or decoder usually stacks several such blocks:

$$X^{(0)} \rightarrow X^{(1)} \rightarrow X^{(2)} \rightarrow \dots \rightarrow X^{(L)}$$

Each block refines the sequence representation.

Intuition:

- early layers capture simpler relations,
- deeper layers capture more abstract relations,
- the final layer supports the prediction task.

What Enters the First Block?

At the beginning of the model, the input is:

$$X^{(0)} = \text{token embeddings} + \text{positional encodings}$$

So the full initial pipeline is:

token IDs $\rightarrow$ embedding lookup $\rightarrow$ add positions $\rightarrow$ Transformer block

This connects directly with the earlier lectures on embeddings and positional information.

From Block Output to Prediction

After the last Transformer block, each position has a contextual representation.

To make predictions, the model usually applies a final linear layer:

$$\text{logits} = HW_{\text{out}} + b$$

where:

- H is the final hidden representation,
- the output dimension equals the vocabulary size (for language modeling).

What Are Logits?

Logits are the raw, unnormalized scores produced by the model before softmax.

If the vocabulary has size $|V|$, then at a given position the model outputs:

$$z \in \mathbb{R}^{|V|}$$

where each component z_c is the score for token class c .

Softmax converts logits into probabilities:

$$p_c = \frac{e^{z_c}}{\sum_u e^{z_u}}$$

Training Objective

For autoregressive language modelling, the model is trained to predict the next token.

Example:

$$(w_1, w_2, w_3, w_4) \rightarrow (w_2, w_3, w_4, w_5)$$

So at each position, the target is the next token.

The model produces logits, then probabilities, then compares them to the true target.

Training process

Fundamental idea

We train tasks, not embeddings directly.

The fundamental pipeline

word indices $\Rightarrow$ embeddings lookup $\Rightarrow$ vectors $\Rightarrow$
self-attention $\Rightarrow$ output $\Rightarrow$ loss $\mathcal{L}(\cdot)$ computation

Training process

- The model makes a prediction
- The prediction is initially wrong
- The loss gradient is computed
- Backpropagation computes:

$$\nabla_{\mathbf{v}_t} \mathcal{L} = \frac{\partial \mathcal{L}}{\partial \mathbf{v}_t} \in \mathbb{R}^K$$

- All model parameters (including embeddings) are updated by gradient descent over many iterations.

Gradient Descent Intuition

At each training step:

- 1 the model computes predictions,
- 2 the loss measures the error,
- 3 backpropagation computes gradients,
- 4 parameters are updated to reduce the loss.

Typical update rule:

$$\theta \leftarrow \theta - \eta \nabla_{\theta} L$$

where:

- θ denotes model parameters,
- η is the learning rate.

Training process-II

Loss function and model's prediction

- There is a loss function $\mathcal{L}(\text{prediction}, \text{target})$ so that *lower loss* $\Rightarrow$ *better prediction*
- When using self-attention, dot products affect weights

$$w_{ij} = \text{softmax}\left(\frac{q_i \cdot k_j}{\sqrt{d_k}}\right)$$

$$y_i = \sum_j w_{ij} v_j$$

- The attention weights influence the output y_i , which affects the prediction and the loss:

$$V_{cat} \cdot V_{walk} \Rightarrow w_{ij} \Rightarrow y \Rightarrow \mathcal{L}$$

Cross-Entropy Loss

If the target token is class c^* , the cross-entropy loss is:

$$L = -\log p_{c^*}$$

where:

$$p_{c^*} = \frac{e^{z_{c^*}}}{\sum_u e^{z_u}}$$

Interpretation:

- if the correct class gets high probability, the loss is small,
- if the correct class gets low probability, the loss is large.

Training process – III

Gradient descent and the loss

Vectors are updated by gradient descent:

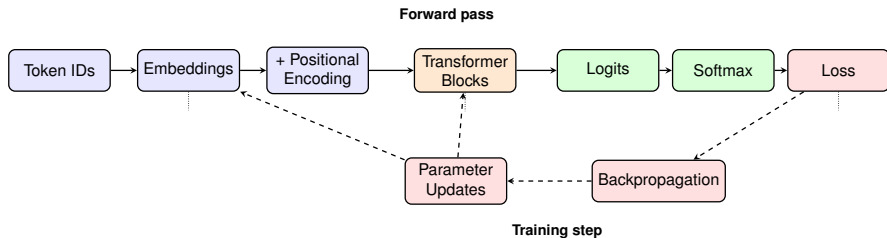
$$\mathbf{v}_{\text{cat}} \leftarrow \mathbf{v}_{\text{cat}} - \eta \frac{\partial \mathcal{L}}{\partial \mathbf{v}_{\text{cat}}} \quad \mathbf{v}_{\text{walks}} \leftarrow \mathbf{v}_{\text{walks}} - \eta \frac{\partial \mathcal{L}}{\partial \mathbf{v}_{\text{walks}}}$$

The dot product changes indirectly because the vectors change
 $\mathbf{v}_{\text{cat}} \cdot \mathbf{v}_{\text{walks}}$ is modified through these updates.

Interpretation

- If the gradient encourages alignment, the dot product increases.
- If the gradient discourages alignment, the dot product decreases.
- The change happens because lowering the loss reshapes the geometry.

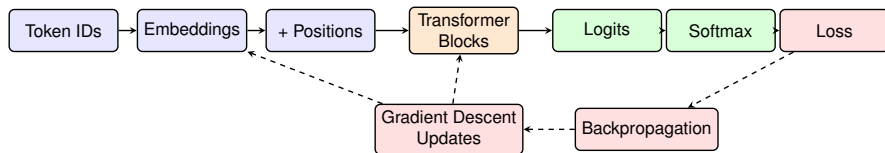
End-to-End Training Pipeline



Forward pass: prediction and loss

Backward pass: gradients and updates

Training Pipeline: Visual Overview



What Parameters Are Learned?

Training updates all learnable parameters, including:

- embedding matrix E ,
- Query, Key, Value projection matrices,
- output projection matrices,
- feed-forward weights,
- normalization parameters,
- final output layer weights.

So the geometry of the representation space is learned jointly with the whole model.

Why Training Changes Geometry

The model is not trained to “make nice embeddings” directly.

Instead:

- it is trained to reduce prediction loss,
- this changes the parameters,
- the dot products and representations change indirectly.

So geometry emerges because it helps solve the task, not because it is explicitly imposed.

Encoder vs Decoder Note

A practical note:

- encoder-style Transformers use full self-attention,
- autoregressive decoders use masked self-attention,
- the block structure is similar, but the attention mask changes.

This will matter in the next block, when we study autoregressive generation.

Where Stability Comes From

The main stability mechanisms in the Transformer block are:

- scaling by $\sqrt{d_k}$ inside attention,
- residual connections,
- layer normalization.

Together, these make it possible to train deep attention-based models more reliably.

Compact Summary of the Full Block

A standard Transformer block can be summarized as:

$$Z_1 = \text{LayerNorm}(X + \text{MHA}(X))$$

$$Y = \text{LayerNorm}(Z_1 + \text{FFN}(Z_1))$$

Training then uses:

$Y \rightarrow \text{logits} \rightarrow \text{softmax} \rightarrow \text{cross-entropy loss}$

followed by backpropagation and gradient descent.

Concept Check

- 1 Why is multi-head attention more expressive than a single head?
- 2 What is the purpose of a residual connection?
- 3 Why is layer normalization useful?
- 4 Why do we need a feed-forward network after attention?
- 5 What are logits?
- 6 What is optimized during training?

Suggested Exercise

Describe the flow for one batch:

- 1 token IDs,
- 2 embedding lookup,
- 3 add positional encoding,
- 4 run one Transformer block,
- 5 compute logits,
- 6 compute cross-entropy loss,
- 7 explain which parameters receive gradients.

This helps connect architecture and optimization.

Self-attention mechanism in transformers

Single-head self-attention (symbolic dimensions)

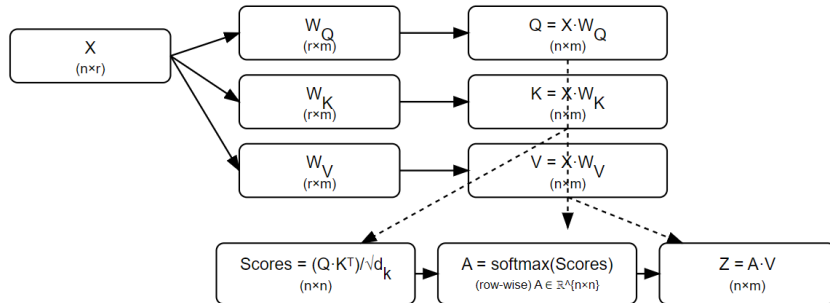


Figure: Self-attention mechanism in Transformers

Toy example (follow up)

"The cat walks on street":

- Let's take embeddings as 2D vectors

	words in sequence	input vector	embeddings
1	the	x_{the}	(0.1, 0.0)
2	cat	x_{cat}	(1.0, 0.9)
3	walks	x_{walks}	(0.9, 1.0)
4	on	x_{on}	(0.0, 0.1)
5	street	x_{street}	(0.8, 0.7)

- Projection matrices W_q , W_k , W_v , we assume the following numeric values:

$$W_Q = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \quad W_K = \begin{pmatrix} 0.8 & 0.2 \\ 0.1 & 0.9 \end{pmatrix} \quad W_V = \begin{pmatrix} 1 & -0.3 \\ 0.2 & 0.7 \end{pmatrix}$$

Toy example-II

$$q_i = W_Q x_i$$

$$q_{\text{the}} = (0.1, 0.0)$$

$$q_{\text{cat}} = (1.0, 0.9)$$

$$q_{\text{walks}} = (0.9, 1.0)$$

$$q_{\text{on}} = (0.0, 0.1)$$

$$q_{\text{street}} = (0.8, 0.7)$$

$$k_i = W_K x_i$$

$$k_{\text{the}} = (0.08, 0.01)$$

$$k_{\text{cat}} = (0.98, 0.91)$$

$$k_{\text{walks}} = (0.92, 0.91)$$

$$k_{\text{on}} = (0.02, 0.09)$$

$$k_{\text{street}} = (0.78, 0.71)$$

$$v_i = W_V x_i$$

$$v_{\text{the}} = (0.1, 0.02)$$

$$v_{\text{cat}} = (0.73, 0.83)$$

$$v_{\text{walks}} = (0.6, 0.88)$$

$$v_{\text{on}} = (-0.03, 0.07)$$

$$v_{\text{street}} = (0.59, 0.65)$$

Scaled dot-product scores for token `cat`

$$s_{\text{cat},j} = \frac{q_{\text{cat}}^\top k_j}{\sqrt{2}}$$

Computed values

$$s_{\text{cat},\text{the}} = 0.06293$$

$$s_{\text{cat},\text{cat}} = 1.27209$$

$$s_{\text{cat},\text{walks}} = 1.28057$$

$$s_{\text{cat},\text{on}} = 0.07142$$

$$s_{\text{cat},\text{street}} = 1.00338$$

Softmax normalization for token `cat`

$$\sum_j e^{s_{\text{cat},j}} = 13.098$$

$$w_{\text{cat},j} = \frac{e^{s_{\text{cat},j}}}{\sum_k e^{s_{\text{cat},k}}}$$

Attention weights

$$w_{\text{cat,the}} = 0.0813$$

$$w_{\text{cat,cat}} = 0.27248$$

$$w_{\text{cat,walks}} = 0.27469$$

$$w_{\text{cat,on}} = 0.081997$$

$$w_{\text{cat,street}} = 0.208199$$

$$\sum_j w_{\text{cat},j} = 1$$

$$y_{\text{cat}} = \sum_j w_{\text{cat},j} v_j = (0.50037, 0.61223)$$

The output is a weighted combination of value vectors, not a probability vector.

Takeaways

- A Transformer block is more than attention alone.
- It combines attention, residual connections, normalization and feed-forward layers.
- Several blocks are stacked to refine sequence representations.
- Training is end-to-end with logits, softmax, cross-entropy and backpropagation.
- The model learns useful geometry because that geometry reduces prediction loss.

Learning Goals

- map the attention equations to PyTorch operations,
- understand tensor dimensions in attention code,
- implement basic self-attention with matrix multiplications,
- implement projected scaled dot-product attention,
- understand the role of batch size, sequence length and model dimension,
- connect attention code with the mathematical formula,
- prepare the ground for a minimal Transformer implementation.

Transition to the Next Block

From:

- embeddings and positional encoding,
- self-attention as weighted sums,
- Queries, Keys and Values,
- scaling by $\sqrt{d_k}$,
- Transformer block structure and training pipeline.

Now we ask:

Main question

How do we implement attention in PyTorch in a way that matches the mathematics exactly?

Attention Formula to Implement

The target formula is:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

Implementation steps:

- 1 compute Queries, Keys and Values,
- 2 compute attention scores,
- 3 scale the scores,
- 4 apply softmax,
- 5 compute weighted sums of the Value vectors.

What a Tensor Represents

In a batched implementation, the input tensor usually has shape:

$$x \in \mathbb{R}^{b \times t \times d_{\text{model}}}$$

where:

- b = batch size,
- t = sequence length,
- d_{model} = embedding/model dimension.

So each batch contains several sequences, and each sequence contains several token vectors.

Dimension Intuition

$$X \in \mathbb{R}^{b \times t \times d_{\text{model}}}$$

b = number of sequences in the batch

t = number of tokens per sequence

d_{model} = vector size for each token

#Token	Value vector (dimension 5) (b=1, t=4, d_model=5)
# The	[0.4, -0.2, 0.8, 0.1, 0.6]
# cat	[0.1, 0.5, -0.3, 0.7, 0.2]
# is	[0.3, -0.4, 0.2, 0.6, 0.5]
# sleeping	[0.9, 0.1, -0.2, 0.3, 0.4]

First Step: Basic Self-Attention Without Projections

Before implementing Q/K/V, we can implement the simpler version:

$$S = XX^{\top}$$

$$W = \text{softmax}(S)$$

$$Y = WX$$

Basic Self-Attention in PyTorch

```
1 import torch
2 import torch.nn.functional as F
3
4 # x has shape (batch_size, seq_len, d_model)
5 x = ...
6
7 # Compute raw attention scores: (b, t, t)
8 raw_scores = torch.bmm(x, x.transpose(1, 2))
9
10 # Optional scaling
11 d = x.size(-1)
12 raw_scores = raw_scores / (d ** 0.5)
13
14 # Normalize across keys
15 weights = F.softmax(raw_scores, dim=2)
16
17 # Weighted sum of values (here values = x)
18 y = torch.bmm(weights, x)
```

How `torch.bmm` Works

`torch.bmm` performs batch matrix multiplication.

If:

$$A \in \mathbb{R}^{b \times m \times n}, \quad B \in \mathbb{R}^{b \times n \times p}$$

then:

$$\text{torch.bmm}(A, B) \in \mathbb{R}^{b \times m \times p}$$

In attention:

$$x \in \mathbb{R}^{b \times t \times d}$$

$$x^T \in \mathbb{R}^{b \times d \times t}$$

so:

$$xx^T \in \mathbb{R}^{b \times t \times t}$$

Why `x.transpose(1, 2)` ?

The input tensor has shape:

$$x \in \mathbb{R}^{b \times t \times d}$$

To compute all pairwise dot products between tokens, we need:

$$x^T \in \mathbb{R}^{b \times d \times t}$$

In PyTorch:

```
x.transpose(1, 2)
```

swaps:

- dimension 1: sequence length
- dimension 2: feature dimension

Dimension Flow of Basic Self-Attention

$$\mathbf{x} \in \mathbb{R}^{b \times t \times d}$$

$$\mathbf{x}^{\top} \in \mathbb{R}^{b \times d \times t}$$

$$\text{raw_scores} = \mathbf{x}\mathbf{x}^{\top} \in \mathbb{R}^{b \times t \times t}$$

$$\text{weights} = \text{softmax}(\text{raw_scores}) \in \mathbb{R}^{b \times t \times t}$$

$$\mathbf{y} = \text{weights} \cdot \mathbf{x} \in \mathbb{R}^{b \times t \times d}$$

So the output has the same outer shape as the input.

What the Score Tensor Means

For each batch element:

$$\text{raw_scores}[i] \in \mathbb{R}^{t \times t}$$

This matrix contains:

- one row per query position,
- one column per candidate key position,
- one score for every token-to-token comparison.

After softmax, each row becomes a probabilistic distribution over the sequence.

What the Score Tensor Means

```
1 # Step 1 -- raw attention scores
2 scores = torch.bmm(x, x.transpose(1,2))
3 Raw scores shape: torch.Size([1, 4, 4])
4 tensor([[[ 1.2100, -0.1100,  0.7200,  0.4500],
5           [-0.1100,  0.8800,  0.2900,  0.4900],
6           [ 0.7200,  0.2900,  0.9000,  0.5700],
7           [ 0.4500,  0.4900,  0.5700,  1.1100]]])
8 # Step 3 -- softmax
9 weights = F.softmax(scores, dim=2)
10 tensor([[[
11           [0.25  0.25  0.25  0.25]
12           [0.30  0.40  0.10  0.20]
13           [0.20  0.30  0.30  0.20]
14           [0.35  0.20  0.15  0.30]]]])
15 # Step 4 -- weighted sum
16 y = torch.bmm(weights, x)
17 Output shape: torch.Size([1, 4, 5])
18 tensor([[[ 0.4356, -0.0564,  0.2124,  0.3856,  0.4552],
19           [ 0.4133,  0.0456,  0.0614,  0.4544,  0.4016],
20           [ 0.4290, -0.0332,  0.1515,  0.4184,  0.4373],
21           [ 0.4590,  0.0048,  0.0989,  0.4186,  0.4231]]]])
```

Second Step: Scaled Dot-Product Attention with Projections

Now move from basic attention to the real Transformer version:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

Then:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

This what we need for a minimal Transformer implementation.

Projected Scaled Dot-Product Attention in PyTorch

```
1 import torch
2 import torch.nn.functional as F
3 from math import sqrt
4 # x : (batch_size, seq_len, d_model)
5 x = ...
6 # projection matrices
7 W_Q = ...
8 W_K = ...
9 W_V = ...
10 Q = x @ W_Q      # (b, t, d_k)
11 K = x @ W_K      # (b, t, d_k)
12 V = x @ W_V      # (b, t, d_v)
13 scores= torch.bmm(Q, K.transpose(1, 2))/sqrt(Q.size(-1))
14 weights = F.softmax(scores, dim=2)
15
16 y = torch.bmm(weights, V)    # (b, t, d_v)
```

Why @ Works Here

If:

$$x \in \mathbb{R}^{b \times t \times d_{\text{model}}}$$

and

$$W_Q \in \mathbb{R}^{d_{\text{model}} \times d_k}$$

then:

$$Q = x @ W_Q \in \mathbb{R}^{b \times t \times d_k}$$

The same applies to:

$$K = x @ W_K, \quad V = x @ W_V$$

So each token vector is projected into a new feature space.

Dimension Flow of Projected Attention

$$X \in \mathbb{R}^{b \times t \times d_{\text{model}}}$$

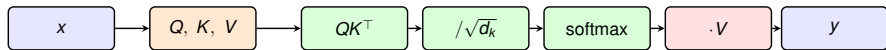
$$Q, K \in \mathbb{R}^{b \times t \times d_k}, \quad V \in \mathbb{R}^{b \times t \times d_v}$$

$$QK^T \in \mathbb{R}^{b \times t \times t}$$

$$\text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) \in \mathbb{R}^{b \times t \times t}$$

$$Y \in \mathbb{R}^{b \times t \times d_v}$$

A Visual Pipeline



Using `nn.Linear` Instead of Raw Matrices

In practice, PyTorch models usually define projections with learnable layers:

$$Q = \text{Linear}_Q(x), \quad K = \text{Linear}_K(x), \quad V = \text{Linear}_V(x)$$

This is equivalent to multiplying by learned matrices plus optional bias terms.

Minimal Attention Module with `nn.Linear`

```
1 import torch          import torch.nn as nn
2 import torch.nn.functional as F
3 from math import sqrt
4 class SimpleAttention(nn.Module):
5     def __init__(self, d_model, d_k, d_v):
6         super().__init__()
7         self.W_Q = nn.Linear(d_model, d_k, bias=False)
8         self.W_K = nn.Linear(d_model, d_k, bias=False)
9         self.W_V = nn.Linear(d_model, d_v, bias=False)
10    def forward(self, x):
11        Q = self.W_Q(x)
12        K = self.W_K(x)
13        V = self.W_V(x)
14        scores= torch.bmm(Q,K.transpose(1,2))/
15        sqrt(Q.size(-1))
16        weights = F.softmax(scores, dim=2)
17        y = torch.bmm(weights, V)
18        return y
```


Why The Minimal Pytorch's Attention Module Is Useful

This module illustrates how to obtain/carry out:

- learnable Query, Key and Value projections,
- batch attention computation,
- scaled dot-product attention,
- output contextualization.

Where Softmax Is Applied

In the code:

```
weights = F.softmax(scores, dim=2)
```

Why `dim=2`?

Because the tensor:

$$\text{scores} \in \mathbb{R}^{b \times t \times t}$$

has:

- one row dimension for query positions,
- one last dimension for candidate key positions.

So softmax must normalize across the key positions.

Fundamental question

Why do we normalize over keys and not over batches or features?

Because for each token position i , we want:

$$w_{i1}, w_{i2}, \dots, w_{it}$$

to form a probability distribution over the tokens it can attend to.
So each row of the attention matrix should sum to 1.

Masked Attention Preview

In autoregressive models, we must prevent a token from attending to future tokens.

That will later be done by modifying the scores before softmax:

$$\text{scores}[\text{masked positions}] = -\infty$$

Then softmax gives zero weight to forbidden positions.

This is the basis of causal masking, which we will study in the next block.

What Masking Looks Like in Code

```
1 mask = torch.triu(torch.ones(t, t), diagonal=1).bool()  
2 scores = scores.masked_fill(mask, float('-inf'))  
3 weights = F.softmax(scores, dim=2)
```

The idea is simple:

- future positions are assigned $-\infty$,
- softmax turns them into zero probability.

Step-by-step breakdown of `mask = torch.triu(...)`

```
torch.ones(t, t)
```

create a matrix full of 1s; example (t=4):

```
[[1, 1, 1, 1],  
 [1, 1, 1, 1],  
 [1, 1, 1, 1],  
 [1, 1, 1, 1]]
```

```
torch.triu(..., diagonal=1)
```

`triu` means “upper triangular” matrix `diagonal=1` means: exclude the main diagonal and keep only future positions. And `.bool()` converts the 1s into boolean mask.

```
torch.triu(torch.ones(t, t), diagonal=1).bool()  
[[False, True, True, True],  
 [False, False, True, True],  
 [False, False, False, True],  
 [False, False, False, False]]
```

Attention and masking

How to block attention

Each row “ i ” in matrix mask represents the current token i

Each column “ j ” in matrix mask represents token j it could attend to

```
mask[i, j] = True -> "block attention"
mask[i, j] = False -> "allow attention"
```

For the token positions: `[[The], [cat], [is], [sleeping]]`

The mask says:

- "The" cannot see $\rightarrow$ cat, is, sleeping
- "cat" cannot see $\rightarrow$ is, sleeping
- "is" cannot see $\rightarrow$ sleeping
- "sleeping" sees everything before

Training Compatibility

The attention code is fully differentiable.

That means PyTorch can automatically compute gradients through:

- matrix multiplications,
- softmax,
- weighted sums,
- linear projections.

So this implementation integrates naturally into end-to-end training.

What Parameters Will Be Learned?

In the attention module above, training will update:

- the weights of W_Q ,
- the weights of W_K ,
- the weights of W_V .

If the model also includes embeddings and output layers, those parameters are trained jointly as well.

Typical Pitfalls in Implementation

Common mistakes:

- forgetting to transpose K ,
- applying softmax over the wrong dimension,
- forgetting the scaling factor,
- using mismatched dimensions for Q , K , V ,
- misunderstanding what shape `torch.bmm` expects.

Debugging by Shapes

A very good practice is to check tensor shapes explicitly.
For example:

- $x: (b, t, d_{\text{model}})$
- $Q: (b, t, d_k)$
- $K^\top: (b, d_k, t)$
- scores: (b, t, t)
- weights: (b, t, t)
- output: (b, t, d_v)

If the shapes make sense, the code is usually close to correct.

Shape Debugging Example

```
1 print("x:", x.shape)
2 print("Q:", Q.shape)
3 print("K:", K.shape)
4 print("V:", V.shape)
5 print("scores:", scores.shape)
6 print("weights:", weights.shape)
7 print("y:", y.shape)
```

This is often the fastest way to diagnose implementation errors.

From Here to a Full Transformer

Once attention is implemented, the next steps are:

- add residual connections,
- add layer normalization,
- add the feed-forward network,
- add masking for autoregressive generation,
- stack several blocks.

So attention code is the computational core of the Transformer.

Compact Summary

Basic self-attention:

$$\text{raw_scores} = \mathbf{x}\mathbf{x}^\top$$

$$\text{weights} = \text{softmax}(\text{raw_scores})$$

$$\mathbf{y} = \text{weights} \mathbf{x}$$

Projected scaled dot-product attention:

$$\mathbf{Q} = \mathbf{x}\mathbf{W}_Q, \quad \mathbf{K} = \mathbf{x}\mathbf{W}_K, \quad \mathbf{V} = \mathbf{x}\mathbf{W}_V$$

$$\mathbf{y} = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right) \mathbf{V}$$

The PyTorch implementation mirrors these equations very directly.

Concept Check

- 1 Why do we use `transpose(1, 2)` on K ?
- 2 What is the shape of the score matrix?
- 3 Why is softmax applied on `dim=2`?
- 4 What does `torch.bmm(weights, V)` compute?
- 5 Why does the implementation remain differentiable?

Suggested Coding Exercise

Implement a function that:

- 1 receives x of shape (b, t, d) ,
- 2 computes Q, K, V ,
- 3 computes scaled attention,
- 4 returns y ,
- 5 prints the intermediate shapes.

This is a perfect precursor to the mini-project.

Takeaways

- PyTorch attention code follows the mathematics very closely.
- `torch.bmm` is the key operation for batched attention.
- Correct tensor dimensions are essential.
- The implementation of Q/K/V and scaling is simple once shapes are clear.
- This block prepares for autoregressive masking and the mini-project.

Transition to the Next Block

Next topic:

How does the Transformer generate sequences autoregressively?
How do we prevent a token from looking into the future?

Next block:

Autoregressive Transformers and causal masking

Main question

How can a Transformer generate a sequence one token at a time without cheating by looking at the future?

What Does Autoregressive Mean?

Autoregressive modelling means:

$$p(w_1, \dots, w_T) = \prod_{t=1}^T p(w_t \mid w_1, \dots, w_{t-1})$$

Interpretation:

- each token is predicted from the previous tokens,
- the model generates the sequence from left to right,
- the future cannot be used to predict the present.

Next-Token Prediction

In autoregressive training, the model receives an input sequence and predicts the next token at every position.

Example:

input: (w_1, w_2, w_3, w_4)

target: (w_2, w_3, w_4, w_5)

So the model learns:

$$w_1 \rightarrow w_2, \quad (w_1, w_2) \rightarrow w_3, \quad (w_1, w_2, w_3) \rightarrow w_4, \dots$$

Why Ordinary Self-Attention Is Not Enough

Standard self-attention lets every token attend to every token:

$$y_i = \sum_{j=1}^t w_{ij} v_j$$

This includes positions with:

$$j > i$$

That would let token i look at future tokens.

Problem

In next-token prediction, the model must not use information from the future.

The Cheating Problem

Suppose the sequence is:

(The, cat, is, sleeping)

If the model tries to predict `sleeping`, it is allowed to use:

(The, cat, is)

but not the token `sleeping` itself.

Without masking, attention would expose future tokens during training and the model would learn an invalid shortcut.

Causal Masking: Main Idea

To prevent attention to future positions, we introduce a mask that blocks all entries above the diagonal.

This means:

- token 1 can only attend to token 1,
- token 2 can attend to tokens 1, 2,
- token 3 can attend to tokens 1, 2, 3,
- and so on.

This is called a **causal mask** or **look-ahead mask**.

Causal Mask Matrix

For a sequence of length 4, the causal mask is:

$$M = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$

Interpretation:

- 1 = attention allowed
- 0 = attention forbidden

This triangular structure enforces left-to-right dependence.

Visual Intuition of the Mask

	1	2	3	4
1				
2				
3				
4				

row = query position

column = attended position

Green cells are allowed. Red cells correspond to future positions and must be masked.

Masked Attention Formula

The standard attention scores are:

$$S = \frac{QK^{\top}}{\sqrt{d_k}}$$

With causal masking, the scores become:

$$S'_{ij} = \begin{cases} S_{ij}, & j \leq i \\ -\infty, & j > i \end{cases}$$

Then attention weights are computed as:

$$W = \text{softmax}(S')$$

Finally:

$$Y = WV$$

Why Use $-\infty$?

The mask is usually applied by replacing forbidden scores with a very negative number:

$$S'_{ij} = -\infty \quad \text{for forbidden positions}$$

Then softmax gives:

$$e^{-\infty} = 0$$

So masked positions receive zero probability.

Key idea

Masking is implemented before softmax, not after.

Masked Softmax Example

Suppose a row of scores is:

$$(1.2, 0.8, 2.0, 1.5)$$

If the last two positions are forbidden, we replace them by:

$$(1.2, 0.8, -\infty, -\infty)$$

After softmax, the weights become:

$$(\alpha, \beta, 0, 0)$$

with:

$$\alpha + \beta = 1$$

So the model attends only to allowed past positions.

How This Changes the Transformer

A decoder-style autoregressive Transformer uses:

- the same Q/K/V mechanism,
- the same scaling by $\sqrt{d_k}$,
- the same residual and normalization structure,
- but with **masked self-attention** instead of full self-attention.

So the block architecture stays similar, but the attention pattern is restricted.

Decoder Attention vs Encoder Attention

Type	Attention pattern	Typical use
Encoder self-attention	all positions visible	classif., encoding
Decoder masked attention	only past and present visible	generation

The key distinction is whether future positions are accessible.

Autoregressive Training Is Parallel

An important observation:

Even though generation is sequential at inference time, training can still be parallelized.

Why?

Because the full input sequence is available during training, and the mask ensures that each position only uses the allowed prefix.

So the model predicts all next tokens in parallel, while preserving the autoregressive constraint.

Training Example

Suppose the input is:

(h, e, l, l, o)

Then the targets are:

$(e, l, l, o, <end>)$

With masking:

- position 1 predicts e from h
- position 2 predicts l from he
- position 3 predicts l from hel
- etc.

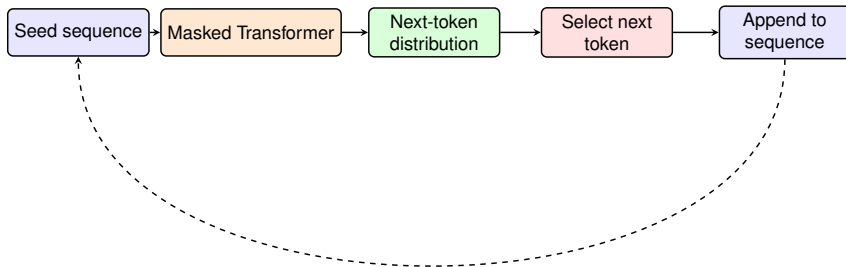
Inference-Time Generation

At inference time, generation proceeds step by step:

- 1 provide a seed sequence,
- 2 run the model,
- 3 predict the next token,
- 4 append it to the sequence,
- 5 repeat.

This is an autoregressive loop.

Generation Loop



Teacher Forcing

During training, autoregressive Transformers usually use **teacher forcing**.

This means:

- the model is given the true prefix,
- not its own previously generated predictions,
- and it learns to predict the next token.

This makes training more stable and more efficient.

What Is Predicted at the Output?

At each position, the final hidden representation is mapped to logits:

$$z_t \in \mathbb{R}^{|V|}$$

Then softmax gives a next-token distribution:

$$p(w_{t+1} \mid w_1, \dots, w_t)$$

Training minimizes cross-entropy with respect to the true next token.

Character-Level Generation

For a character-level model, the vocabulary is a set of characters:

$$V = \{a, b, c, \dots, z, \text{space}, \dots\}$$

The model predicts:

- the next character,
- then the next,
- then the next,

until a desired length or end token is reached.

This is exactly the setup of the mini-project.

PyTorch: Building the Mask

A causal mask for a sequence of length t can be created with an upper triangular matrix.

The forbidden region is the strictly upper triangular part.

In PyTorch, a standard way is:

```
torch.triu(..., diagonal=1)
```

PyTorch Code for the Causal Mask

```
1 import torch
2
3 t = 5
4 mask = torch.triu(torch.ones(t, t), diagonal=1).bool()
5
6 print(mask)
```

This produces a Boolean mask where:

- True marks forbidden future positions,
- False marks allowed positions.

Applying the Mask to Attention Scores

```
1 scores = torch.bmm(Q, K.transpose(1, 2)) / (Q.size(-1) **  
    0.5)  
2  
3 mask = torch.triu(torch.ones(t, t), diagonal=1).bool()  
4 scores = scores.masked_fill(mask, float('-inf'))  
5  
6 weights = torch.softmax(scores, dim=2)  
7 y = torch.bmm(weights, V)
```

This is the standard implementation of masked scaled dot-product attention.

Shape of the Mask

If:

$$\text{scores} \in \mathbb{R}^{b \times t \times t}$$

then the mask typically has shape:

$$(t \times t)$$

PyTorch broadcasts it across the batch dimension.

So the same causal structure is applied to every sequence in the batch.

Why the Mask Is Applied Before Softmax

If we applied the mask after softmax, the remaining weights would no longer sum to 1.

By masking scores before softmax:

- forbidden positions get zero probability,
- allowed positions are still properly normalized,
- each row remains a valid attention distribution.

Sampling at Inference Time

Once the model outputs next-token probabilities, we still need to choose the next token.

Common strategies:

- greedy decoding: choose the most probable token,
- sampling: draw according to the probability distribution,
- temperature scaling: control randomness.

This determines the style and diversity of the generated text.

Temperature

Given logits z , temperature modifies them as:

$$p_i = \text{softmax}\left(\frac{z_i}{T}\right)$$

Interpretation:

- $T < 1$: sharper distribution, more deterministic
- $T > 1$: flatter distribution, more random

This is often used in text generation experiments.

Generation Example

Suppose the seed is:

"The cat"

The model predicts a distribution over the next token:

$p(\text{sat}), p(\text{is}), p(\text{ran}), \dots$

After selecting one token, it is appended and the model is run again on the longer prefix.

So generation is iterative.

Why Autoregressive Transformers Are Powerful

They combine:

- global contextual attention,
- strict causal constraints,
- parallel training,
- sequential generation at inference time.

This is what makes decoder-style Transformers so effective for language modelling and text generation.

Fundamental question

If training is parallel, why do we say generation is sequential?

Because:

- during training, the full sequence is known and masking enforces valid dependencies,
- during inference, future tokens do not exist yet, so the model must generate them one by one.

Autoregressive Transformer Block Summary

A masked decoder block uses:

- 1 input embeddings + positions,
- 2 masked multi-head self-attention,
- 3 residual connection + normalization,
- 4 feed-forward network,
- 5 output projection to logits.

Then the model is trained with next-token cross-entropy loss.

Compact Summary of the Core Mechanism

Masked attention:

$$S = \frac{QK^\top}{\sqrt{d_k}}$$

$$S'_{ij} = \begin{cases} S_{ij}, & j \leq i \\ -\infty, & j > i \end{cases}$$

$$W = \text{softmax}(S')$$

$$Y = WV$$

This is the mathematical core of autoregressive Transformers.

Concept Check

- 1 Why is ordinary self-attention invalid for next-token prediction?
- 2 What does a causal mask forbid?
- 3 Why is the mask applied before softmax?
- 4 Why can training still be parallelized?
- 5 Why is generation sequential at inference time?

Suggested Exercise

- 1 write the causal mask for a sequence of length 5,
- 2 show which positions token 4 can attend to,
- 3 explain how the mask changes the score matrix,
- 4 describe one iteration of the generation loop.

Takeaways

- Autoregressive Transformers model the next token from the past context only.
- Causal masking prevents future information leakage.
- The mask is implemented by setting forbidden scores to $-\infty$ before softmax.
- Training is parallel over all positions, but generation is sequential.
- This is the mechanism behind modern Transformer-based text generation.